

DEEP LEARNING FINAL PROJECT REPORT

Fine-Grained Image Recognition under Limited Training Data

CUB-200-2011 Bird Species Classification

Group ID: 6

Project ID: 2

University of Science and Technology of Hanoi (USTH)

Academic Year 2026-2027

Submission file name: Group6_Project2_Report.pdf

Abstract

Fine-grained image recognition is difficult because different categories can share very similar visual features. The problem becomes more challenging when only a limited number of labeled training images are available. This project studies how training data size, transfer learning, and data augmentation affect bird-species classification on CUB-200-2011. The dataset contains 11,788 images from 200 bird species. We train a simple CNN from scratch as a baseline and compare it with ImageNet-pretrained ResNet18 using a frozen backbone and partial fine-tuning. We also train the baseline with 5, 10, 15, and 20 images per class and evaluate a basic augmentation pipeline. The experiments show that increasing the amount of training data generally improves the baseline. Transfer-learning models perform much better than the scratch baseline in our setup, and ResNet18 with partial fine-tuning obtains the best test accuracy of 55.26%, compared with 50.88% for frozen ResNet18 and 4.42% for BaselineCNN. Basic augmentation changes baseline accuracy from 4.42% to 4.57%, which is too small to support a strong improvement claim. These results highlight the usefulness of pretrained representations while also showing the difficulty of fine-grained classification under limited labeled data.

1. Introduction

Image classification is a common computer-vision task, but recognizing closely related categories is more difficult than distinguishing broad object groups. In fine-grained image recognition (FGIR), classes may share similar shapes, colors, and textures, and the useful differences can be small. Bird species are a typical example: several species can look similar at first sight and may differ only in details of the beak, wing, body pattern, or feather color.

A second challenge is the limited availability of labeled training data. Deep models normally benefit from large datasets, while collecting and labeling many examples for every fine-grained class can be expensive. When a model has only a small number of examples per class, it can overfit or fail to learn enough discriminative features. This motivates studying not only model architecture but also the effect of training-data size and training strategy.

In this project, we use CUB-200-2011, a benchmark containing 11,788 images of 200 bird species [1]. We first train a small CNN from scratch as a baseline. We then use ImageNet-pretrained ResNet18 [2], [3] with two strategies: a frozen backbone and partial fine-tuning. In addition, we change the number of training images per class and test a basic augmentation pipeline.

The project focuses on the following research questions:

- RQ1: How does the amount of labeled training data affect fine-grained bird-classification accuracy?
- RQ2: How effective is transfer learning compared with a CNN trained from scratch under limited labeled data?
- RQ3: How does basic data augmentation affect the generalization of the baseline CNN?

The goal is not to propose a state-of-the-art FGIR model. Instead, the project uses controlled experiments to understand the behavior and limitations of several practical approaches under the same dataset setting.

2. Related Work

Convolutional neural networks (CNNs) learn hierarchical image representations directly from pixels. Early convolutional layers usually respond to simple patterns such as edges and textures, while deeper layers combine these patterns into more task-specific features. However, a CNN trained from random initialization must learn all useful representations from the target dataset, which is difficult when labeled data is limited.

Residual Networks were proposed to make deeper neural networks easier to optimize by learning residual functions with shortcut connections [2]. In a residual block, the transformed features are added to the block input. ResNet18 is a relatively compact member of this family and is available with ImageNet-pretrained weights. ImageNet provides large-scale supervised visual pretraining and has been widely used as a source for transfer learning [3].

Transfer learning reuses knowledge from a source task or domain for a new target task [4]. In this project, ImageNet is the source of pretrained visual features and CUB-200-2011 is the target classification task. With a frozen strategy, pretrained backbone weights are kept fixed and only a new classifier is learned. With partial fine-tuning, later layers are also updated, allowing high-level features to adapt to the target bird classes.

Data augmentation is a common approach for training with limited image data. It exposes a model to transformed versions of training images and can reduce dependence on a particular pose, crop, or appearance [5]. However, augmentation does not create independent real-world labeled observations. Its benefit depends on the selected transformations, model, dataset, and training setup.

3. Dataset and Data Preparation

3.1 CUB-200-2011

CUB-200-2011 is a fine-grained bird dataset with 11,788 images covering 200 species [1]. The official dataset provides image paths, class labels, class names, and a predefined train/test split. The official source used by the project is CaltechDATA (<https://data.caltech.edu/records/65de6-vp158>).

Item	Value
Total images	11,788
Classes	200
Official training images	5,994
Official test images	5,794
Final training split	4,794
Validation split	1,200
Random seed	42

Table 1. Dataset statistics used in the project.

3.2 Train, Validation, and Test Splits

The official test split is preserved for final evaluation. The official training portion contains 5,994 images. The project creates a validation subset from this portion, producing 4,794 training images and 1,200 validation images. Training data is used to update model parameters, validation data is used during development and checkpoint selection, and the official test split is used to report final performance.

3.3 Preprocessing and Class Labels

Image preprocessing and augmentation are centralized in the common data pipeline. Validation, test, and inference use deterministic transformations, while random augmentation is applied only to training images in the augmentation experiment. The original CUB class IDs run from 1 to 200, while PyTorch classification outputs are zero-indexed; therefore, the pipeline converts labels to indices 0-199.

3.4 Limited-Data Sampling

To answer RQ1, the training subset is reduced to 5, 10, 15, or 20 images per class. Validation and test data remain unchanged. Keeping the evaluation splits fixed is important because otherwise changes in accuracy could be caused by different test examples instead of the amount of training data. A fixed seed is used to make the sampling reproducible.

4. Methodology

4.1 BaselineCNN

The baseline is a small convolutional neural network trained from scratch. It contains convolution blocks with Batch Normalization, ReLU activation, and pooling, followed by adaptive average pooling, dropout, and a final linear classifier for 200 classes. The model starts from randomly initialized parameters and therefore must learn all useful representations from the CUB training data. Its purpose is to provide a simple reference point for the later experiments.

4.2 ResNet18 with Frozen Backbone

For the first transfer-learning strategy, the project loads ResNet18 with ImageNet-pretrained weights. Backbone parameters are frozen and the original classification layer is replaced with a new fully connected layer for the 200 bird classes. Only the new classifier is trained. This greatly reduces the number of trainable parameters and retains the pretrained visual representation.

4.3 ResNet18 Partial Fine-Tuning

The second strategy starts from the same pretrained ResNet18 but allows the final residual stage (`layer4`) and the new classification layer (`fc`) to be updated. Earlier layers remain frozen. This design preserves general pretrained features while allowing higher-level representations to adapt to differences between bird species. The best checkpoint from this strategy is also used for the inference demo.

4.4 Augmentation and Limited-Data Training

For RQ3, the same BaselineCNN is trained with the project's basic augmentation pipeline and compared with the baseline without augmentation. For RQ1, the baseline architecture is kept the same while the number of training images per class is changed. These choices keep the experiments simple and make the role of each tested factor easier to interpret.

5. Experimental Setup

The experiments are organized around the three research questions. All final metrics are computed on the same official CUB test split. The project uses a fixed seed of 42 for reproducibility. The main experiments are summarized in Table 2.

Experiment	Models / settings	Purpose	RQ
Transfer learning	BaselineCNN; Frozen ResNet18; Partial ResNet18	Compare scratch and pretrained strategies	RQ2
Training-data size	5, 10, 15, 20 images/class	Measure effect of available labeled data	RQ1
Augmentation	BaselineCNN vs BaselineCNN + basic augmentation	Measure effect of augmentation	RQ3

Table 2. Experimental design.

5.1 Evaluation Metrics

Top-1 accuracy is the main metric: a prediction is correct when the class with the highest model score matches the ground-truth label. Precision, recall, F1-score, and confusion matrices are also generated. Macro-averaged metrics first compute a metric for each class and then average all classes equally, while weighted averages give more weight to classes with more test examples.

Accuracy is easy to interpret, but macro precision, recall, and F1 are useful in a 200-class problem because they provide a class-level view instead of allowing high-support classes to dominate the summary metric.

6. Results and Discussion

6.1 Main Model Comparison

Model / setting	Accuracy	Macro Precision	Macro Recall	Macro F1
BaselineCNN	4.42%	-	-	-
ResNet18 Frozen	50.88%	55.40%	51.23%	51.40%
ResNet18 Partial Fine-Tuning	55.26%	58.84%	55.49%	54.47%
BaselineCNN + Augmentation	4.57%	5.02%	4.68%	3.36%
5 images/class	1.85%	1.72%	1.94%	1.18%
10 images/class	3.07%	2.81%	3.12%	2.10%
15 images/class	3.38%	2.55%	3.44%	2.37%
20 images/class	4.90%	4.16%	4.95%	3.67%

Table 3. Verified final test results.

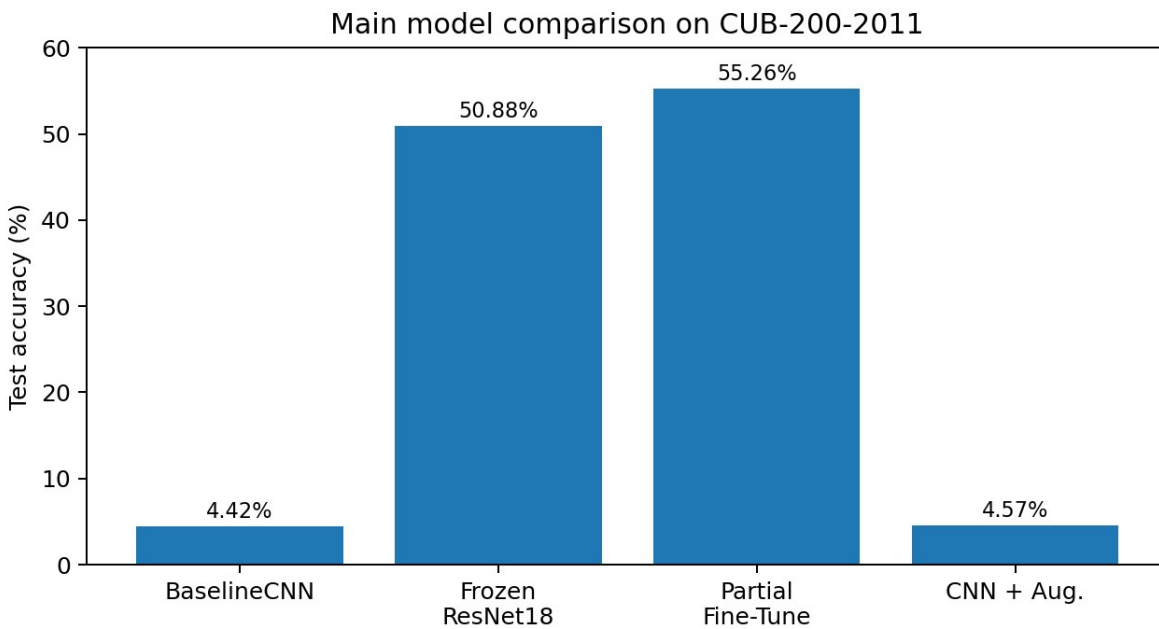


Figure 1. Main test-accuracy comparison.

The best model is ResNet18 with partial fine-tuning at 55.26% test accuracy. Frozen ResNet18 achieves 50.88%, while BaselineCNN reaches 4.42%. The large difference shows that the pretrained ResNet18 approaches are much more effective in this project setting. However, the scratch baseline and ResNet18 also use different architectures, so the full gap cannot be attributed only to ImageNet pretraining.

6.2 RQ1 - Effect of Training Data Size

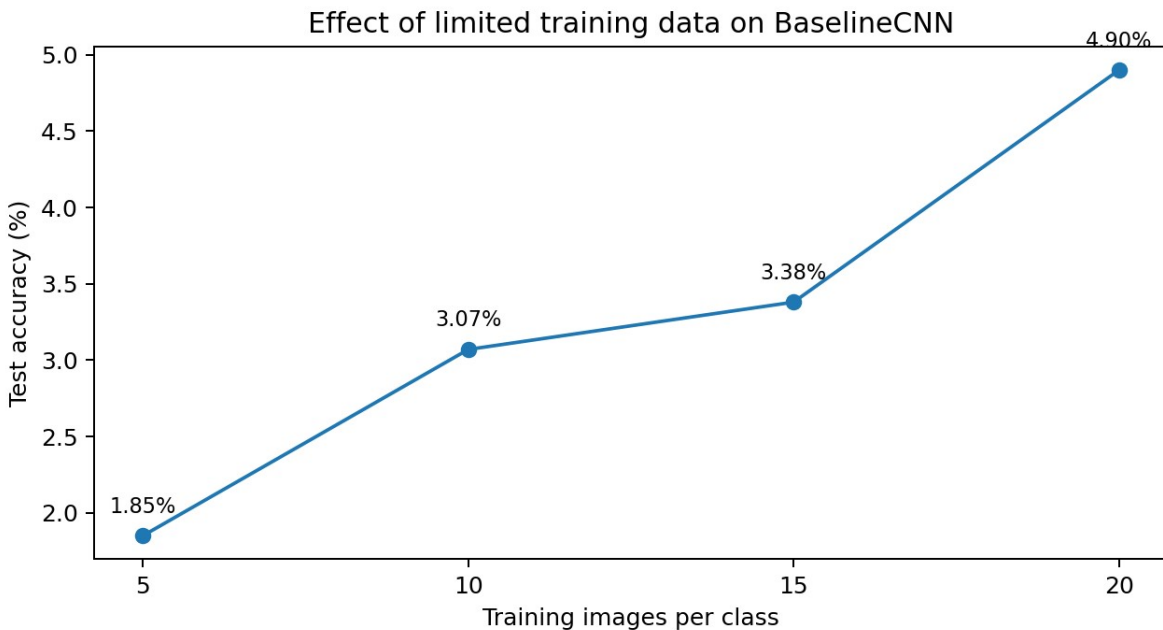


Figure 2. Test accuracy as training samples per class increase.

With 5, 10, 15, and 20 images per class, BaselineCNN obtains 1.85%, 3.07%, 3.38%, and 4.90% test accuracy, respectively. The overall trend is upward, which supports the conclusion that more labeled training examples help the scratch baseline. The improvement is not linear, and only four data-size settings were tested, so the result should be interpreted as an observed trend rather than a universal law.

6.3 RQ2 - Transfer Learning

Both pretrained ResNet18 strategies substantially outperform the scratch baseline in this experiment. A cleaner comparison is between the two ResNet18 strategies because they share the same architecture. Partial fine-tuning improves accuracy from 50.88% to 55.26%, a gain of 4.38 percentage points. This suggests that adapting later pretrained layers to CUB is useful, while keeping early layers fixed still preserves general visual features.

6.4 RQ3 - Data Augmentation

Basic augmentation changes BaselineCNN accuracy from 4.42% to 4.57%, an increase of only 0.15 percentage points. This is a very small difference. The project therefore does not claim that the chosen augmentation policy strongly improves performance. Repeating experiments with several random seeds and testing alternative augmentation policies would be needed to determine whether the effect is stable.

7. Error and Qualitative Analysis

The partial fine-tuned ResNet18 still misclassifies many test images despite being the strongest model. Fine-grained classification is difficult because related species can have similar body shapes, colors, and feather patterns, while discriminative details may be small or difficult to observe. Background, pose, lighting, and object scale can also make classification harder. Without an interpretability method such as Grad-CAM, these factors should be treated as plausible explanations rather than direct evidence of what the network used for a particular prediction.

The team also performed several informal inference tests on images outside the official benchmark. These examples are not included in the official accuracy and are used only to illustrate model behavior.

Input	True / intended category	Top-1 prediction	Observation
External Red-winged Blackbird	Red-winged Blackbird	Red-winged Blackbird (52.64%)	Correct external example
External Blue Jay	Blue Jay	Green Jay (6.00%)	True class appeared at Top-2 (4.94%)
External Bald Eagle	Outside CUB 200 classes	Pomarine Jaeger (3.15%)	Closed-set model cannot output Bald Eagle
Cat image	Non-bird	Carolina Wren (5.26%)	Closed-set model still chooses a bird class

Table 4. Informal qualitative inference examples (not official test metrics).

The Bald Eagle and cat examples show an important limitation: the classifier is closed-set. Its output layer contains only the 200 CUB bird classes, so it always assigns probability to those classes even when the input is an unknown species or is not a bird. A future system could add out-of-distribution detection, open-set recognition, or a separate bird/non-bird detector.

8. Reproducibility and Inference Demo

The repository contains scripts for dataset preparation, training, evaluation, limited-data experiments, model export, and inference. Random seed utilities and configuration files are used to improve reproducibility. The best partial fine-tuning model was exported as a complete ResNet18 state dictionary so that inference does not require downloading ImageNet weights.

The full inference checkpoint is distributed through the project GitHub Release as ``resnet18_partial_full.pth`` (approximately 43.1 MB). The inference script reads the fixed 200-class list from ``configs/classes.txt``, so a user can run prediction on a new image without downloading the complete CUB dataset. The model outputs the Top-k predicted species and softmax scores.

Example command:

```
python -m scripts.predict --image "bird.jpg" --checkpoint  
"checkpoints/resnet18_partial_full.pth" --top-k 5
```

The demo is intended to show how the trained model can be reused without retraining. It does not change the official research results, which are based on the defined CUB test split.

9. Conclusion and Limitations

This project investigated fine-grained bird classification under limited labeled data using CUB-200-2011. For RQ1, increasing the number of training images per class generally improved BaselineCNN performance, although the absolute accuracy remained low. For RQ2, pretrained ResNet18 models were much stronger in this setup, and partial fine-tuning achieved the highest test accuracy of 55.26%, compared with 50.88% for the frozen strategy. For RQ3, basic augmentation changed baseline accuracy only from 4.42% to 4.57%, so the experiment does not provide strong evidence of a substantial augmentation benefit.

Several limitations should be considered. First, BaselineCNN and ResNet18 differ in both architecture and pretraining, so their full performance gap is not a perfectly controlled measurement of the pretraining effect. Second, most settings were not repeated with multiple random seeds, which limits conclusions about statistical stability. Third, only one basic augmentation policy was evaluated. Finally, the model is a closed-set classifier and cannot reject unknown bird species or non-bird images.

Future work could compare pretrained and randomly initialized versions of the same architecture, repeat experiments across multiple seeds, evaluate stronger augmentation strategies, investigate staged or full fine-tuning, and add open-set or out-of-distribution detection. More specialized fine-grained methods could also be explored if the goal is to maximize CUB classification performance.

References

- [1] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200-2011 Dataset," California Institute of Technology, Technical Report CNS-TR-2011-001, 2011.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770-778, 2016.
- [3] O. Russakovsky et al., "ImageNet Large Scale Visual Recognition Challenge," International Journal of Computer Vision, vol. 115, no. 3, pp. 211-252, 2015.
- [4] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," IEEE Transactions on Knowledge and Data Engineering, vol. 22, no. 10, pp. 1345-1359, 2010.
- [5] C. Shorten and T. M. Khoshgoftaar, "A survey on Image Data Augmentation for Deep Learning," Journal of Big Data, vol. 6, article 60, 2019.

Appendix A. Team Contributions

Member	Main contribution
Nguyen Trung Hieu	Repository setup/integration; BaselineCNN pipeline; configuration and reproducibility utilities; experiment integration; inference/export workflow.
Hoang Gia Khanh	Dataset selection/documentation and dataset preparation/DataLoader work.
Pham Quang Anh	ResNet18 transfer-learning implementation and experiments.
Tran Duc Anh	Data-augmentation experiment.
Tran Xuan Bac	Limited-data sampling and sampling tests.
Lam Pham	Evaluation/results-analysis utilities.
Do Thanh Tung	Slides and presentation preparation.

Table A1. Working contribution summary. Each member should confirm before final submission.

Appendix B. Repository Reproduction Summary

1. Clone the repository and use the develop/final submission branch as instructed by the group.
2. Create a Python virtual environment and install dependencies from requirements.txt.
3. For training/evaluation, download and prepare CUB-200-2011 according to DATA.md.
4. Run the provided training scripts for baseline, transfer learning, augmentation, and data-size experiments.
5. Use the evaluation script to reproduce test metrics.
6. For inference only, download the full checkpoint from the GitHub Release and run scripts.predict with a new image.